

$x_s^{(n)} = \{x_\lambda : \lambda \in s^{(n)}\} \subset x^{(n)}$ be the subset of features, which represents the selected and revealed features. Our task is to select features $s^{(n)}$ and make a final prediction about $y^{(n)}$ while $s^{(n)}$ is different for each instance.

Our purpose is to find two functions, implemented as neural networks; the policy to select features and the predictor to make predictions. The policy takes the revealed features $x_s^{(n)}$ and the whole features of features $z^{(n)}$ as input. Then, it returns one index and the corresponding feature will be selected and revealed. We denote the policy as $\pi(x_s^{(n)}, z^{(n)}) \in \Lambda^{(n)}$. The predictor also takes $x_s^{(n)}$ and $z^{(n)}$ as input and it returns the prediction about $y^{(n)}$. We denote the prediction and the predictor as $\hat{y}^{(n)} = f(x_s^{(n)}, z^{(n)})$.

We innovate a way to select features one by one with the policy until the predetermined number of features is selected. Firstly, we determine the budget k , which is the number of selected features in the end. In the initial state, we suppose $s^{(n)} = \phi$ and $x_s^{(n)} = \phi$, that is, any features has not revealed. Note that, we assume that features of features z are fully revealed in the initial state and they are given as prior information. This assumption is natural because we consider features of features as the description or the property of each feature on the natural language or numerical expression and they are usually given before the feature values are revealed. Then, we select a feature with the policy $\pi(x_s, z)$ and reveal the selected feature. (In other words, $s \leftarrow s \cup \pi(x_s, z)$ and the size of x_s increases by one.) We repeat this process for k times and make the final prediction with the predictor $f(x_s, z)$ from $|x_s| = k$ features. This procedure is shown in Fig. 1b and 1c.

Our ultimate goal is to minimize the loss of the prediction made by the predictor after selecting features by the policy. Given the loss function $l(\hat{y}, y)$, the objective function of our problem is

$$\min_{\theta, \phi} \mathbb{E}_{\mathbf{y}, \mathbf{x}} l(f_\theta(\mathbf{x}_s, z), \mathbf{y}), \quad (1)$$

where θ and ϕ are parameters for parameterized predictor and policy respectively. In addition, s ($|s| = k$) is constructed by the iterative selection of the policy $\pi_\phi(x_s, z)$.

4 Greedy Dynamic Feature Selection

This section explains the existing DFS method based on conditional mutual information (CMI) [3]. Note that this method handles the special case of our problem definition that the feature set is fixed. Namely, $\Lambda = [d]$ and $\Lambda^{(n)} = \Lambda$ for all $n = 1, \dots, N$. Therefore, in this section, we note $x = \{x_1, \dots, x_d\}$ and $x_s = \{x_i : i \in s \subset [d]\}$. Also, the policy $\pi(x_s)$ and the predictor $f(x_s)$ take only revealed features x_s as input.

4.1 Greedy Dynamic Feature Selection

This method aims to find a greedy policy based on CMI and the predictor based on the naive Bayesian, implemented as neural networks such as MLP. Concretely,